

Exploratory Data Analysis for Big Data

A Student Guide for Grading, Coding Style, and Project Milestones

Mahmoud Harding

2026-08-10

Table of contents

Preface	4
Introduction	5
R Project Folder	6
Setting up Your RProject Folder	6
Step 1: Create a new RStudio project	6
Step 2: Create the data folder	10
Step 3: Download the <code>us_farmers_markets.csv</code> data file	11
Step 4: Download the <code>.Rmd</code> file	12
Step 5: Always open the <code>.Rproj</code> file	14
Knitting to PDF	15
Knitting Assignment 00	15
Knitting to PDF	15
Using the YAML Header	15
Using the Knit Menu	16
Common Knitting Errors	16
Missing <code>knitr</code>	16
Missing LaTeX	16
Installing TinyTeX for PDF Output	17
TinyTeX vs. <code>tinytex</code>	17
Steps to Set Up TinyTeX	17
Step 1: Install the <code>tinytex</code> Package	17
Step 2: Install TinyTeX	18
Step 3: Verify the Installation	18
Step 4: Knit to PDF	18
Tag System Overview	19
Purpose	19
Tag System Overview	19
Severity Tags	19
Content Tags	20
Multiple Tags and One Deduction	20

Programming Style Guide	21
Purpose	21
General Guidelines	21
Comments	21
Pick clear, descriptive names and a standard naming convention	21
Write code in logical blocks	21
Use consistent spacing	22
Handle long lines thoughtfully	22
Import libraries and load packages at the beginning	23
Python-Specific Style	23
Chain methods when appropriate	23
Define functions clearly	23
R-Specific Style	23
Use the Pipe >	23
Format pipes with whitespace	24
Format <code>ggplot2</code> code	24
Use <code><-</code> for assignment	24
Formatting if statements	24
References	25

Preface



This resource is intended for students in *Exploratory Data Analysis for Big Data*. It provides materials and guidance to support understanding the grading rubrics, coding style, and project milestones.

Introduction

This resource is intended for students in *Exploratory Data Analysis for Big Data*. It provides materials and guidance to support success throughout the course, including information about grading rubrics, coding style expectations, assignment requirements, and project milestones.

The guide is designed to help students understand how coursework will be evaluated and how to develop clear, readable, and well-documented code in both R and Python. It also outlines expectations for visualizations, written interpretations, and project milestones.

R Project Folder

Setting up Your RProject Folder

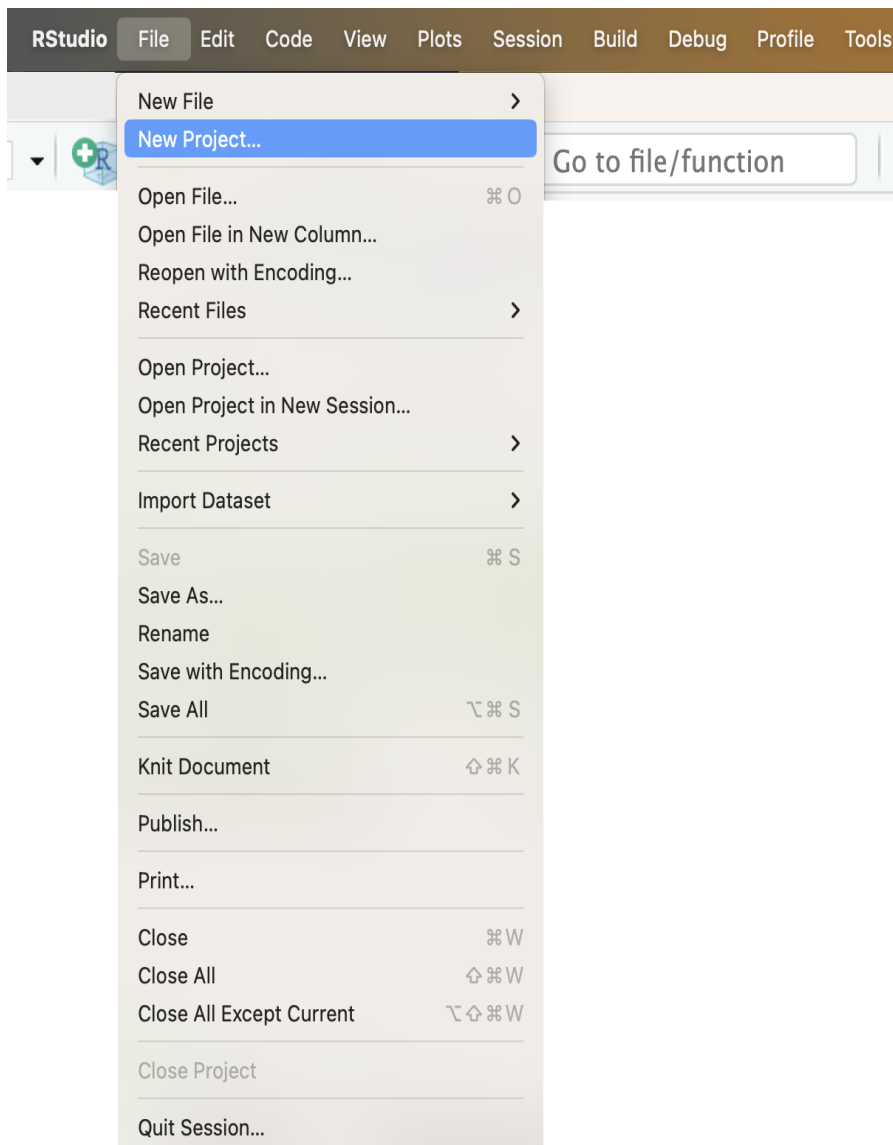
Follow these steps to:

- Create an Rproject folder named `dsc201`.
- Create a data folder inside the RProject folder.
- Download a dataset the `us_farmers_markets.csv` into the data folder.
- Download the `ds201_001-002_a00.Rmd` RMarkdown file into the RProject folder.

Note: Following these steps helps avoid file path errors and follows best practices from [Martin Chan's RStudio Projects and Working Directories: A Beginner's Guide](#).

Step 1: Create a new RStudio project

1. Open RStudio.
2. Go to **File** → **New Project...**



3. Select **New Directory** → **New Project**.

New Project Wizard

Create Project



New Directory

Start a project in a brand new working directory



Existing Directory

Associate a project with an existing working directory



Version Control

Checkout a project from a version control repository



Cancel

4. **Select New Project.**

New Project Wizard

Back

Project Type

 New Project >

 R Package >

 Shiny Application >

 Quarto Project >

 Quarto Website >

 Quarto Blog >

 Quarto Book >

Cancel

5. In **Directory name:**, enter dsc201 (all lowercase)
6. In Create project as subdirectory of:, select Browse, then choose a location on your computer (e.g., Documents, Desktop, etc).

New Project Wizard

[Back](#) **Create Quarto Project**



Directory name:

Create project as subdirectory of:
 [Browse...](#)

Engine: ▾

☐ Create a git repository

☐ Use renv with this project

☐ Use visual markdown editor [?](#)

☐ Open in new session

[Create Project](#) [Cancel](#)

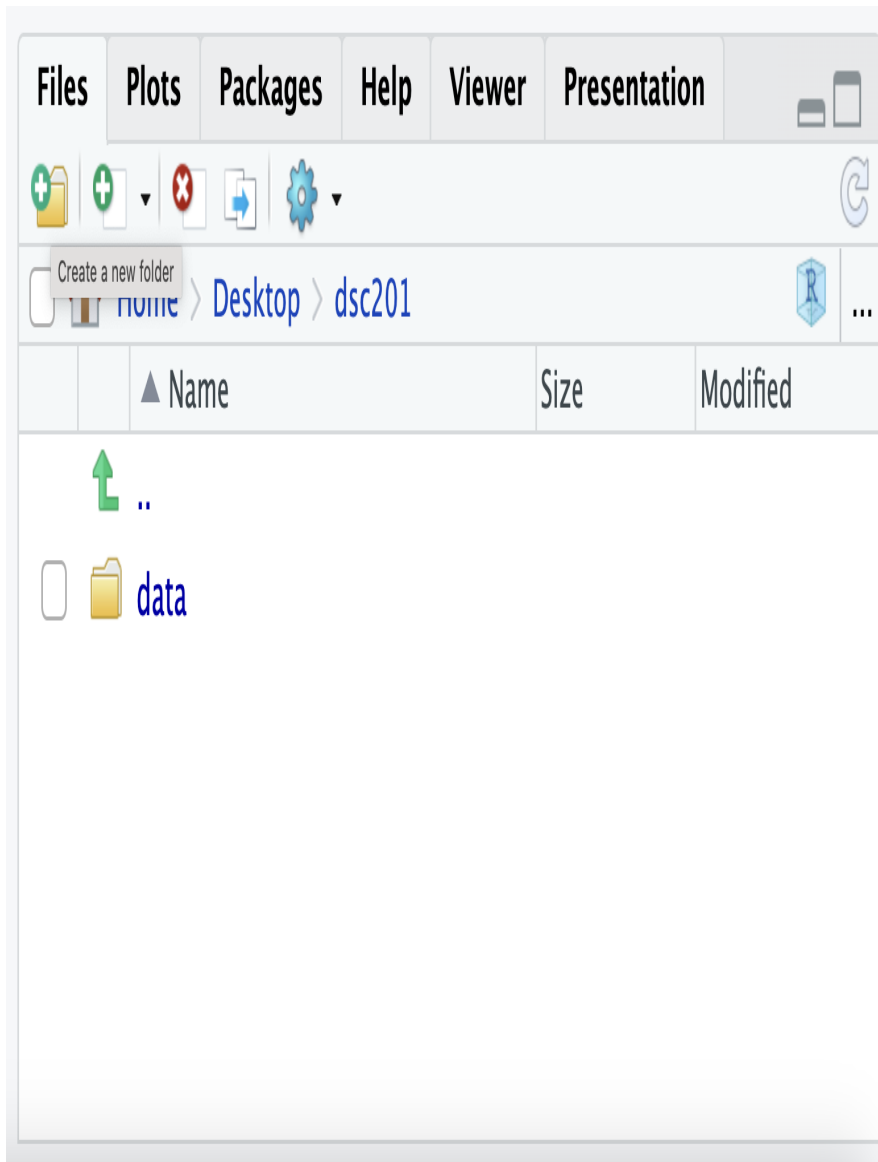
7. Click **Create Project**.

Note: Opening the `dsc201.Rproj` file will automatically set your working directory to the project's root folder.

Step 2: Create the data folder

Important: Name the folder `data` (all lowercase).

8. In RStudio's **Files** pane (bottom right), click the **New Folder** icon.



9. In **Please enter the new folder name**, enter data.

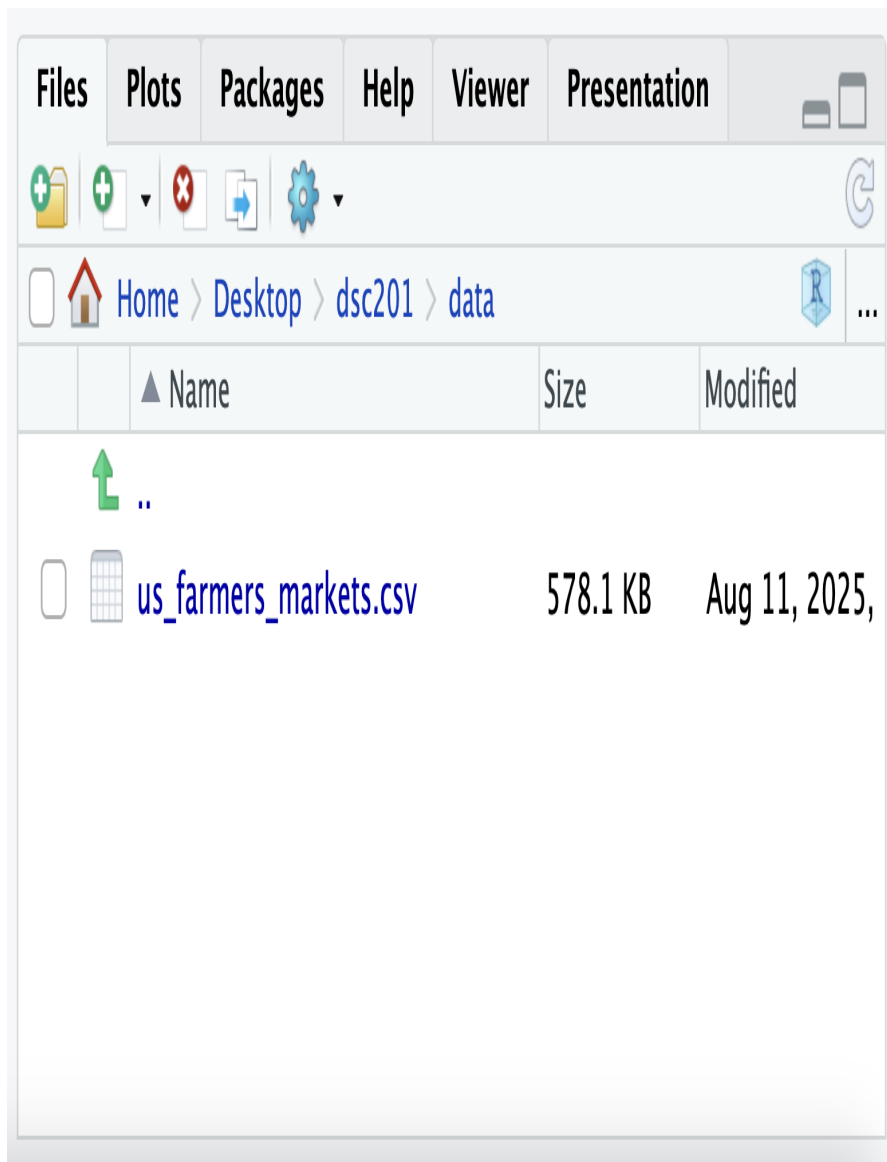
10. Click **OK**.

Important: Name the folder **data** (**all lowercase**). Using all lowercase avoids issues on operating systems where **Data** and **data** are treated as different folders.

Step 3: Download the `us_farmers_markets.csv` data file

11. Download the `us_farmers_markets.csv` dataset from the course Moodle page.

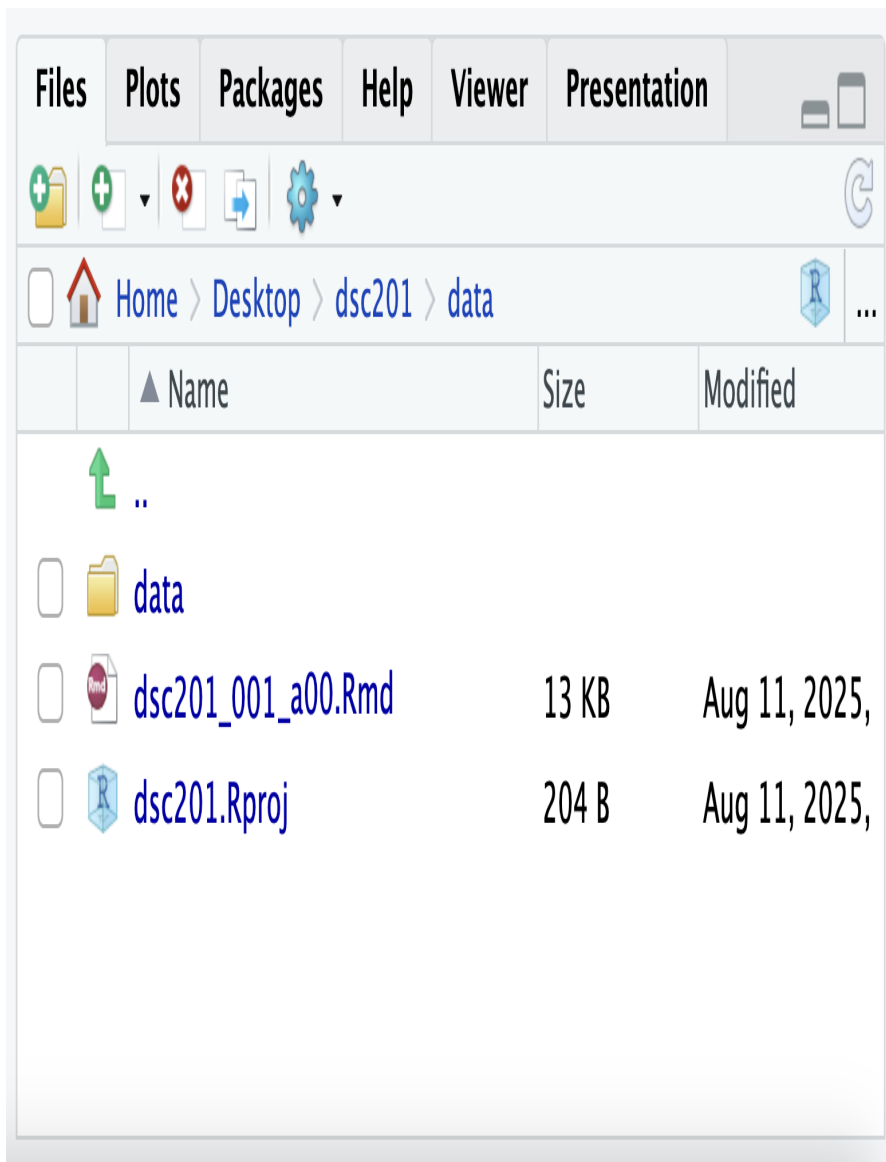
12. Locate it in your browser's **Downloads** folder then move it manually into your `dsc201/data` RProject folder.
13. Confirm the file is in the right place by checking in RStudio's **Files** pane. You should see it in `data/us_farmers_markets.csv`.



Step 4: Download the .Rmd file

14. Download the `dsc201_001_a00.Rmd` RMarkdown file from the course Moodle page.

15. Locate it in your browser's **Downloads** folder then move it to your **dsc201** RProject folder (the root folder, not inside **data**).
16. Confirm the file is in the right place by checking in RStudio's **Files** pane. You should see it in **dsc201/ds201_001_a00.Rmd**.



Step 5: Always open the .Rproj file

17. In your computer's file explorer, double-click `dsc201.Rproj` to start each session. This ensures your working directory and relative paths work correctly.

Knitting to PDF

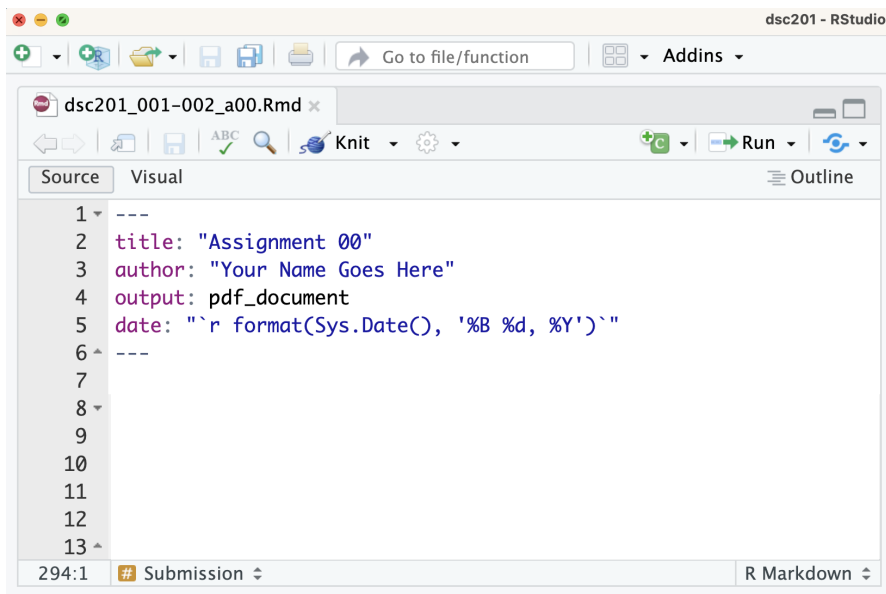
Knitting Assignment 00

In RStudio, **knitting** is the process of taking a source document written in R Markdown (.Rmd) and combining the text, code, and code output into a single rendered file, such as HTML, PDF, or Word. When a document is knitted, RStudio runs all code chunks, captures their output (e.g., tables, plots, and summaries), and integrates the results with the written text.

Knitting to PDF

Using the YAML Header

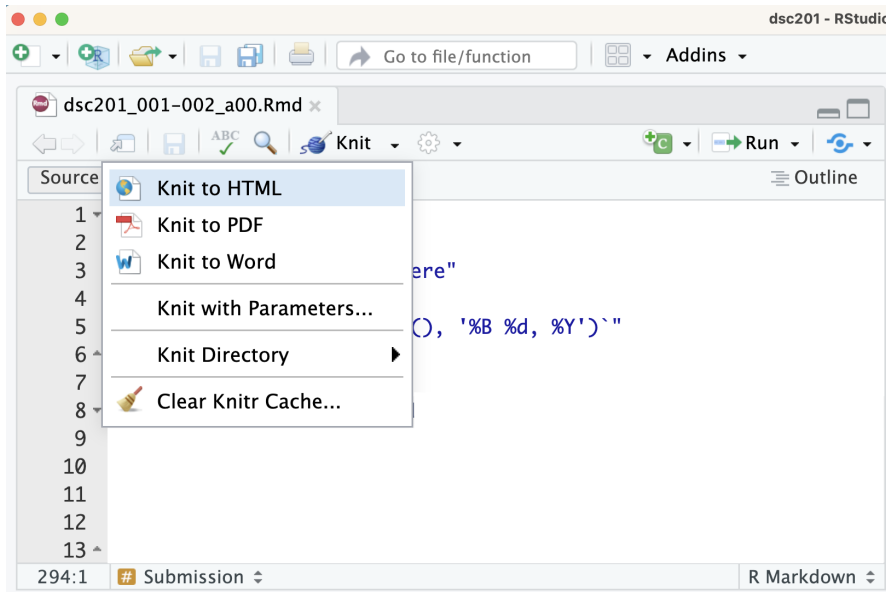
To knit directly to PDF by default, specify PDF output in the YAML header at the top of your document.



Using the Knit Menu

To manually knit a document to PDF:

1. Click the **Knit** button in RStudio.
2. Select **Knit to PDF** from the drop-down menu.



Common Knitting Errors

Missing knitr

If you see the following message when attempting to knit

Required package 'knitr' is missing. Would you like to install it?

the **knitr** package has not yet been installed on your computer.

Click **Yes** and RStudio will automatically install **knitr** and any required dependencies. Once installation is complete, knitting should begin again automatically.

If the document knits successfully, no further action is required.

Missing LaTeX

If you instead see an error similar to the one below:

Warning

Error: LaTeX failed to compile RMarkdown.tex. See <https://yihui.org/tinytex/r/#debugging> for debugging tips. In addition: Warning message: In system2(..., stdout = if (use_file_stdout()) f1 else FALSE, stderr = f2) : ‘“pdflatex”’ not found
Execution halted No LaTeX installation detected (LaTeX is required to create PDF output). You should install a LaTeX distribution for your platform: <https://www.latex-project.org/get/>

your computer does not have a LaTeX distribution installed. To create PDF files from R Markdown, you must install a LaTeX distribution such as TinyTeX.

Proceed to the instructions below.

Installing TinyTeX for PDF Output

To knit R Markdown documents to PDF, your computer needs a LaTeX installation. We recommend TinyTeX because it is lightweight, easy to install, and integrates well with RStudio.

TinyTeX vs. `tinytex`

TinyTeX

TinyTeX is a lightweight LaTeX distribution used to compile R Markdown documents into PDF files. It performs the actual PDF creation process.

`tinytex`

`tinytex` is an R package that installs and manages TinyTeX from within RStudio.

Steps to Set Up TinyTeX

Step 1: Install the `tinytex` Package

- Open RStudio.
- In the **Console**, run:

```
install.packages("tinytex")
```

Note: This installs the `tinytex` package, which manages the TinyTeX installation.

Step 2: Install TinyTeX

- In the **Console**, run:

```
tinytex::install_tinytex()
```

Note: This command downloads and installs TinyTeX on your computer and may take several minutes.

Step 3: Verify the Installation

- In the **Console**, run:

```
tinytex::tinytex_root()
```

If the command returns a file path, TinyTeX has been installed successfully.

Example paths:

- macOS/Linux: `"/usr/local/texlive/..."`
- Windows: `"C:\\Users\\YourName\\AppData\\Roaming\\TinyTeX\\..."`

Step 4: Knit to PDF

- Open your `.Rmd` file in RStudio.
- Verify that the YAML header specifies PDF output:

```
---  
title: "Your Title"  
author: "Your Name"  
output: pdf_document  
---
```

- Click **Knit**.
- Select **Knit to PDF** if prompted.

RStudio will now use TinyTeX to generate a PDF version of your document.

Tag System Overview

Purpose

This section explains the tag system used in grading and feedback for activity assignments, programming assignments, and final project milestones.

The goal of this system is to provide transparent, consistent, and instructional feedback that helps you understand how your work was evaluated and how to improve.

Tag System Overview

Assignment comments may include two kinds of tags:

- **Severity tags** describe how serious the issue is and determine the point deduction.
- **Content tags** describe what kind of issue occurred.

Tags appear at the start of comments in this format:

[SEVERITY] [CONTENT-TAG]

For example:

[S] [CODE-REQ]

This means the response has a major issue related to the code that was submitted to answer the question.

Severity Tags

Severity tags indicate the overall impact of an issue on correctness or completion. The severity tag determines the point deduction.

Multiple content tags may apply to a response, but only one severity level is assigned.

Content Tags

Content tags identify issues related to the completeness, correctness, clarity, interpretation, organization, or communication of a response.

The specific content tags available differ by assignment type.

Multiple Tags and One Deduction

Multiple tags may apply to the same response. However, deductions reflect the overall severity of the issue rather than treating each tag as a separate deduction.

Content tags identify the specific issues present, while the severity tag determines the point deduction.

For example, consider the following question:

Calculate the average calories for each restaurant and identify the restaurant with the highest average calories.

A student submits:

```
ms2022 |>  
  count(restaurant)
```

Possible tags:

[S] [CODE-REQ] [CODE-RESULT] [REQ-INCOMP].

- [S] because most required elements are missing or incorrect.
- [CODE-REQ] because the required computation is missing.
- [CODE-RESULT] because the result does not support the analysis requested.
- [REQ-INCOMP] because the question was not fully answered.

Although multiple content tags apply, the response receives one severity level based on the overall quality of the answer.

Programming Style Guide

Purpose

The purpose for this guide is to help you write clear, readable, and professional code from the start. It draws on recommendations from Pruim et al. (2023), Rossum et al. (2001), and Wickham (2015).

General Guidelines

Comments

- Comments start with #
- Comments should be sentence case

```
# Load the data
```

- Comments should usually be a short phrase or single sentence that describes the purpose of a block of code. Use additional sentences only to explain non-obvious decisions (e.g., `# Keep only data from 2024. Earlier years use a different reporting system and are not directly comparable.`)

Pick clear, descriptive names and a standard naming convention

- Use meaningful names (e.g., `average_score`, not `x1`)
- Use `snake_case` for variable and user-defined function names
- Avoid using dots in names (e.g., `average.score`)

Write code in logical blocks

- Use blank lines between steps

```
# Load the data
code

# Clean the data
code
code
code
```

Use consistent spacing

For assignment statements and arithmetic expressions.

Lan- guage	Good Example	Bad Example
R	<code>height <- feet * 12 + inches</code>	<code>height<-feet*12+inches</code>
Python	<code>df["lbs"] = df["kg"] / 2.204</code>	<code>df["lbs"]=df["kg"]/2.204</code>

For assigning default values or passing keyword arguments.

Lan- guage	Good Example	Bad Example
R	<code>mean(x, na.rm=TRUE)</code>	<code>mean(x, na.rm = TRUE)</code>
Python	<code>round(number, ndigits=2)</code>	<code>round(number, ndigits = 2)</code>

Handle long lines thoughtfully

- Keep lines 79 chars
- Use parentheses for long expressions

```
df |>
  summarise(
    Sepal.Length = mean(Sepal.Length),
    Sepal.Width = mean(Sepal.Width),
    .by = Species
  )
```

```
results = pd.concat(  
    [df1, df2, df3],  
    ignore_index=True,  
    sort=False  
)
```

Import libraries and load packages at the beginning

```
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt
```

```
library(tidyverse)  
library(ggplot2)
```

Python-Specific Style

Chain methods when appropriate

```
df.groupby("category").mean()
```

Define functions clearly

```
def calculate_mean(values):  
    """Return the mean of a list of numbers."""  
    return sum(values) / len(values)
```

R-Specific Style

Use the Pipe |>

```
data |>  
  filter(score > 80) |>  
  summarise(avg = mean(score))
```

Format pipes with whitespace

- Pipes go at end of line
- Indent by 2 spaces
- Short pipes go on a single line `iris |> summary()`

Format ggplot2 code

```
ggplot(iris, aes(x = Sepal.Width, y = Sepal.Length)) +  
  geom_point() +  
  labs(  
    x = "Sepal width (cm)",  
    y = "Sepal length (cm)",  
    title = "Sepal Dimensions by Species"  
  )
```

Use <- for assignment

```
x <- 10
```

Formatting if statements

```
score <- 10  
if (score > 15) {  
  print("High")  
} else if (score < 15) {  
  print("Medium")  
} else {  
  print("Low")  
}
```


References

- Pruim, Randall, Maria-Cristiana Gîrjău, and Nicholas J. Horton. 2023. “Fostering Better Coding Practices for Data Scientists.” *Harvard Data Science Review* 5 (3). <https://doi.org/10.1162/99608f92.97c9f60f>.
- Rossum, Guido van, Barry Warsaw, and Alyssa Coghlan. 2001. *PEP 8 – Style Guide for Python Code*. <https://peps.python.org/pep-0008/>.
- Wickham, Hadley. 2015. *The Tidyverse Style Guide*. <https://style.tidyverse.org/>.